

LECTURE NOTES

Algorithm and Programming

Program Control Selection and Repetition

Pandu Dwi Luhur Pambudi, S.Kom., M.Kom., M.I.M

04

Learning Outcomes

1. LO2: Analyze programs to solve problems using the C language and AI problem-solving sessions related to the topics.

Outline Materi

1. Pemilihan (Selection)
2. Perulangan (Repetition)
3. Break Vs. Continue
4. Go to dan Label
5. Error Type
6. AI Problem-Solving Session: Program Control Debugging

4.1

Pemilihan (Selection)

Struktur kendali pemilihan digunakan untuk memilih langkah-langkah mana yang akan dikerjakan terlebih dahulu berdasarkan hasil pemeriksaan suatu kondisi. Bahasa C menyediakan 5 langkah dalam pemilihan yaitu:

A. If

If digunakan untuk memilih apakah proses tersebut akan dilakukan atau tidak. Berikut bentuk dari syntax if yang digunakan dalam bahasa C :

```
if (boolean expression) statement;
or
if (boolean expression) {
    statement1;
    statement2;
    .....
}
```

Block of statements

Jika ekspresi boolean menghasilkan benar, maka pernyataan atau beberapa pernyataan akan dieksekusi.

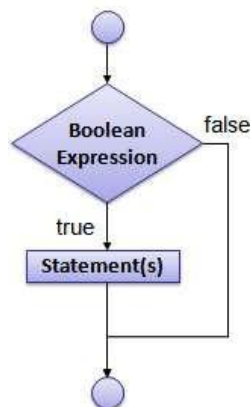
Expression adalah syarat yang akan dikerjakan, syarat ini memiliki nilai benar (true) atau salah (false). Bila *expression* tersebut bernilai benar (true), *statement* di dalam blok if tersebut akan dijalankan. Bila nilai *expression* bernilai salah (false), maka program akan berakhir. Berikut

```
if(r>=0)
{
    area = r*r*pi;
    printf("Area dari r : %d adalah %f",r, area);
}
```

merupakan contoh program if pada bahasa C:

Pada contoh potongan program diatas, misalkan nilai dari radius adalah 5. Program akan mengeksekusi perintah yang ada di blok if tersebut bila syarat terpenuhi. Pertama-tama akan dilakukan pengecekan apakah nilai radius saat ini bernilai lebih dari atau sama dengan 0? Jawabannya adalah benar, $5 \geq 0$, maka program akan menghitung nilai dari area, dan kemudian akan dicetak nilai dari radius dan area tersebut. Bila tidak, maka tidak ada *statement* yang akan dieksekusi(dijalankan).

Secara flowChart bentuk dari if adalah :



B. If-else

If-else digunakan untuk memilih satu dari 2 proses yang akan dieksekusi. Berikut bentuk dari syntax if-else yang digunakan dalam bahasa C:

```

if (boolean expression) statement1;
else statement2;
or
if (boolean expression){
    statement1;
    statement2;
    .....
}
else {
    statement3;
    statement4;
    ...
}
  
```

} Block statement1

} Block statement2

Jika ekspresi boolean menghasilkan benar, maka statement1 atau blok statement1 akan dieksekusi, jika false maka statement2 atau blok statement2 dieksekusi.

Statement untuk kondisi yang benar diletakkan didalam blok if yang dibatasi dengan tanda kurung kurawal buka dan tutup. Kemudian untuk statement kondisi yang salah diletakkan pada else blok yang diawali dan diakhiri dengan kurung kurawal buka dan tutup juga. Berikut contoh program dari penggunaan if-else pada bahasa C:

```
#include <stdio.h>

int main()
{
    int angka;
    scanf("%d", &angka);
    if(angka<0 || angka>100)
        printf("Wrong Input");
    else
        printf("Input : %d", angka);

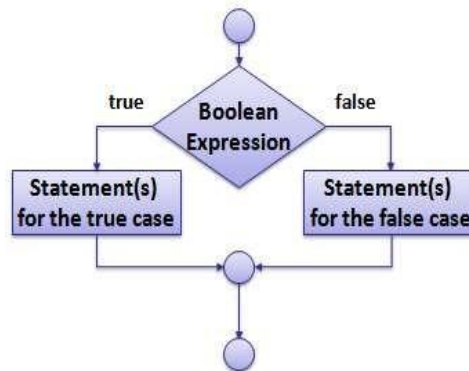
    getchar();
    return 0;
}
```

Pada program tersebut meminta inputan yang akan ditampung pada variabel angka. Fungsi seleksi dilakukan dimana terdapat dua buah kondisi yang dihubungkan dengan logical operator OR “||”, yang berarti bahwa bila salah satu kondisi terpenuhi atau benar maka expression tersebut akan bernilai benar. Andaikan angka bernilai 200, maka program akan mengecek apakah angka < 0? Jawabannya adalah salah (false), kemudian program akan mengecek kembali kondisi berikutnya apakah angka > 100? 200 bernilai lebih dari 100, maka kondisinya adalah benar. Karena relasi yang ada adalah OR, maka nilai akhir dari syarat ini adalah benar, sehingga program akan mencetak “Wrong input”.

Tetapi sebaliknya bila kondisi angka yang dimasukkan adalah 50, maka kondisi pertama dan kedua bernilai salah, dan syarat tidak terpenuhi, maka program akan mencetak “Input : “ yang dilanjutkan dengan nilai dari angka tersebut.

Bila melihat kembali pada program yang ada, pada blok statement if ataupun else tidak terdapat tanda kurung kurawal buka dan tutup “{“ “}” yang biasanya digunakan untuk menandakan batasan blok. Hal ini dapat dilakukan jika hanya ada satu statement yang akan dieksekusi pada blok tersebut. Bila terdapat lebih dari satu statement yang ingin dilakukan pada blok statement if, maka perlu digunakan tanda kurung kurawal buka dan tutup “{“ “}” sebagai penanda.

Secara flow Chart bentuk dari if-else adalah :



C. Nested-IF

Nested if merupakan jenis variasi yang digunakan bila memiliki lebih dari satu syarat dan syarat tersebut saling berkaitan. Yang dimasuk dengan nested adalah di dalam if atau if-else statement terdapat if atau if-else statement lainnya. Tidak ada batasan maximum untuk if di dalam if tersebut. Berikut adalah contoh nested if:

<pre> if (score >= 90) grade = 'A'; else if (score >= 80) grade = 'B'; else if (score >= 70) grade = 'C'; else if (score >= 60) grade = 'D'; else grade = 'F'; </pre>	<pre> if (score >= 90) grade = 'A'; else if (score >= 80) grade = 'B'; else if (score >= 70) grade = 'C'; else if (score >= 60) grade = 'D'; else grade = 'F'; </pre>
---	---

Potongan coding diatas adalah contoh penyeleksian dalam memberikan grade berdasarkan dari nilai yang ada. Bila kondisi tidak dipenuhi maka didalam blok else terdapat kondisi lainnya yang dicek kembali, dan seterusnya. Ini adalah salah satu contoh sederhana dari nested if. Potongan coding dikolom sebelah kiri dan kanan adalah sama, hanya lebih disarankan untuk mengikuti penulisan coding di kolom sebelah kanan.

D. Switch-case

Switch – case adalah salah satu cara seleksi yang digunakan untuk memilih satu dari sejumlah alternatif. Seleksi yang dilakukan pada switch statement berdasarkan pada sebuah status. Format penulisan statement switch itu sendiri dapat dilihat dibawah ini :

```
switch (switch-expression)
```

```
{
```

```
case value1:  statement(s)1;
```

```
break;
```

```
case value2:  statement(s)2;
```

```
break;
```

```
...
```

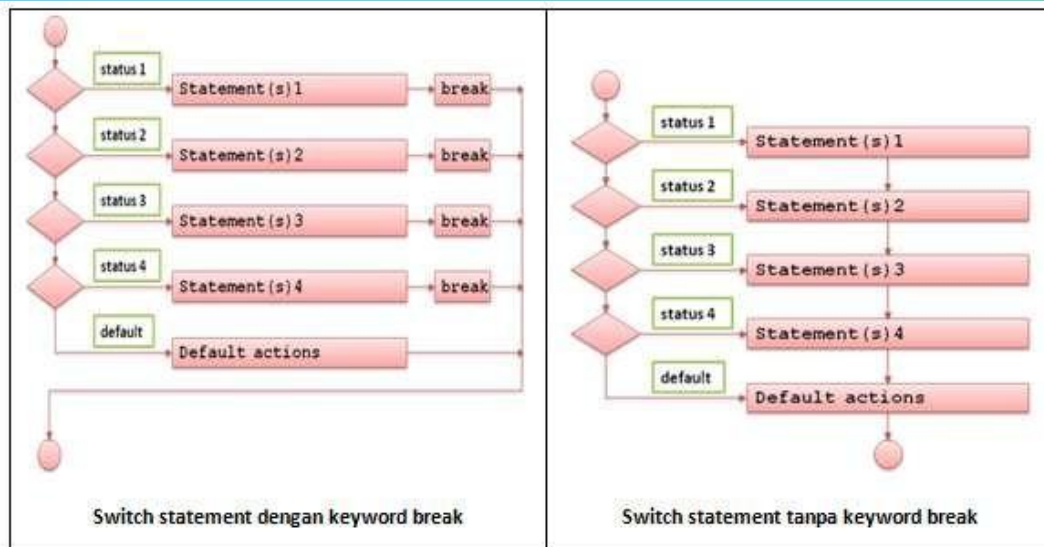
```
case valueN:  statement(s)N;
```

```
break;
```

```
default: statement(s)-for-default;
```

```
}
```

switch-expression hanya boleh menggunakan sebuah nilai dengan tipe karakter(char), byte, short atau int. Kemudian untuk nilai value1, value2, valueN harus mempunyai tipe data yang sama seperti tipe data yang ada di switch-expression. Statement- statement pada setiap pilihan (case) akan dieksekusi sampai program membaca keyword break (akan dijelaskan lebih rinci di topik selanjutnya), atau hingga switch statement telah berakhir. Keyword break dan default case bersifat optional, boleh digunakan boleh juga tidak. Case statement tersebut juga dieksekusi secara berurut (sequential).



Gunakan keyword break bila memang diperlukan. Karena bila tidak ada keyword break, misalnya status 2 terpenuhi, maka statement 2 dan seterusnya akan dieksekusi juga selama blok switch belum berakhir. Untuk lebih jelas dapat dilihat pada flow chart diatas.

Dibawah ini adalah contoh program sederhana yang menerapkan switch statement dalam melakukan seleksi. Seleksi dilakukan terhadap hasil input user berupa bilangan integer yang disimpan ke dalam variable number. Yang menjadi switch expression adalah nilai sisa dari number dibagi dua (modulus). Statement di case 0 akan dijalankan bila sisa bagi bernilai 0, dan statement di case 1 akan dijalankan bila sisa bagi tersebut bernilai 1. Di dalam setiap statement case juga diberikan keyword break. Program sederhana ini merupakan contoh dari penentuan apakah sebuah angka merupakan bilangan ganjil atau genap.

```
#include <stdio.h>

int main()
{
    int number;
    scanf("%d", &number);
    switch(number%2)
    {
        case 0 : printf("%d is even number",number);
                break;
        case 1 : printf("%d is odd number",number);
                break;
    }
    getchar();
    return 0;
}
```

Berikut adalah contoh input dan output dari program tersebut:

```
9
9 is odd number
```

```
10
10 is even number
```

Contoh

```
#include <stdio.h>

int main()
{
    float val1, val2; char op;

    while(1) {

        printf("\n Type val1 operator val2 \n");

        scanf("%f %c %f", &val1, &op, &val2);

        switch(op){

            case('+'): printf(" = %f", val1 + val2); break;

            case('-') : printf(" = %f", val1 - val2); break;

            case('*') : printf(" = %f", val1 * val2); break;

            case('/') : printf(" = %f", val1 / val2); break;

            default : printf(" unknown operator!");

        }

    }

    return(0);
}
```

Note: **case** '+' can also written **case** '+'

Conditional expression

Operator ?: mirip dengan pernyataan if, tetapi mengembalikan nilai. Jenis terakhir dari seleksi statement dapat juga dilakukan dengan bentuk conditional expression. Format yang digunakan adalah sebagai berikut:

Boolean-expression ? expression1 : expression2;

Lihatlah contoh if sederhana yang dituliskan dalam bentuk conditional expression

berikut ini:



Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa mengenai:

Pertanyaan Diskusi:

Bagaimana cara menulis kondisi pada `if` dan `if-else` agar program tetap mudah dibaca dan tidak menimbulkan bug? Berikan contoh penulisan yang baik.

Hint:

- Diskusikan pentingnya penulisan ekspresi boolean yang jelas.
- Bahas penggunaan tanda kurung kurawal `{ }` terutama saat ada lebih dari satu statement di blok `if` atau `else`.
- Bandingkan penulisan `if (a > b)` dengan nested `if`, dan contoh `if-else` bersarang.
- Sertakan ide bagaimana menulis logika yang sederhana sebelum memecah jadi nested `if`.

Pertanyaan Diskusi:

Kapan sebaiknya menggunakan `if-else`, `nested-if`, atau `switch-case` dalam sebuah program C? Berikan contoh kasus nyata.

Hint:

- Diskusikan situasi di mana `if-else` cocok (misalnya hanya dua kondisi).
- Jelaskan kapan `nested-if` bermanfaat (banyak kondisi saling bergantung).
- Jelaskan kapan `switch-case` lebih efisien (banyak alternatif nilai diskrit).
- Berikan contoh: validasi grade nilai, menu input, atau kalkulator sederhana.

Pertanyaan Diskusi:

Bagaimana perbedaan penggunaan `if` biasa dengan conditional expression `? :` dalam program C? Apa keuntungan dan keterbatasannya?

Hint:

- Bahas sintaks `exp1 ? exp2 : exp3;` sebagai versi ringkas dari `if-else`.
- Diskusikan situasi kapan ternary operator lebih praktis.
- Diskusikan juga kapan sebaiknya tetap pakai `if-else` untuk kejelasan program.
- Contoh praktis: menentukan nilai max antara dua bilangan, penetapan grade sederhana.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (If Selection Structure: [Youtube](#)).

4.2

Perulangan (Repetition)

Struktur kendali perulangan atau iterasi (repetition) digunakan untuk melaksanakan satu atau beberapa instruksi secara berulang-ulang. Frekuensi pengulangan dapat ditentukan dalam program atau ditentukan saat program dijalankan.

Dalam iterasi ini instruksi yang dikerjakan berulang-ulang harus memiliki nilai variabel pengendali kondisi yang harus selalu berubah setiap iterasi terjadi agar iterasi dapat berakhir. Pada bahasa C menyediakan 3 instruksi iterasi yaitu :

A. for

For digunakan untuk mengulang suatu statement yang dideklarasikan berdasarkan suatu kondisi atau syarat tertentu.

Format penggunaan For:

- a. Jika hanya 1 statement yang dijalankan dalam perulangan

```
for ([initialization];[condition];[increment/decrement])
    <statement>
```
- b. Jika terdiri lebih dari 1 statement yang dijalankan dalam perulangan

```
for (initialization; condition; increment/decrement)
{
    <block of statements>
}
```

Contoh

```
for ( int i = 0; i < 5; i++) {
    printf("Value of i is = %d" , i);
    printf("Loop number %d", (i+1));
}
```

Pada potongan program di atas, kedua statement yang terdapat didalam for akan dijalankan selama kondisi $i < 5$ terpenuhi. Dalam hal ini, awalnya nilai i adalah 0 dan ketika proses dilakukan, nilai i akan ditambahkan satu point (increment $i++$), dan proses dilakukan terus

B. while

While digunakan untuk mengulang suatu statement selama expression yang diberikan didalamnya bernilai benar, jika bernilai salah, maka statement yang dideklarasikan di dalamnya tidak akan diulang.

```
while (expression) { <block statement> }
```

contoh

```
int i = 0;
while (i < 5){
    printf("Value of i is %d", i);
    i++;
}
```

Pada potongan program di atas, program akan melakukan pengecekan terlebih dahulu apakah nilai $i < 5$, jika benar, maka program akan mengulang kedua statement yang terdapat di dalam while.

C. do-while

Do...While digunakan untuk mengulang suatu statement yang dideklarasikan sesudah keyword Do dan setelah menjalankan statement tersebut, dilakukan pengecekan terhadap expression yang diberikan, apakah bernilai benar atau salah. Jika bernilai benar, maka statement tersebut akan diulang kembali. Jika bernilai salah, maka statement tersebut hanya akan dijalankan satu kali tanpa dilakukan pengulangan.

```
do{ <block statement> }while (expression);
```

contoh

```
int i = 0;
do{
    printf("Value of i is %d", i);
    i++;
} while (i < 5);
```

Pada potongan program di atas, program akan menjalankan terlebih dahulu kedua statement di dalam do kemudian dilakukan pengecekan apakah nilai $i < 5$, jika benar, maka program akan kembali mengulang kedua statement tersebut.

Bahan Diskusi



Pertanyaan Diskusi:

Apa perbedaan mendasar antara struktur perulangan `for`, `while`, dan `do-while` dalam bahasa C? Kapan sebaiknya menggunakan masing-masing?

Hint:

- Bahas perbedaan *alur eksekusi*: `for` cocok untuk pengulangan dengan jumlah iterasi yang sudah jelas, `while` cocok untuk kondisi tak pasti dengan pengecekan di awal, `do-while` cocok jika minimal satu eksekusi tetap dijalankan.
- Diskusikan situasi praktis: `for` untuk array, `while` untuk input validasi, `do-while` untuk menu program.

Pertanyaan Diskusi:

Mengapa variabel pengendali iterasi sangat penting dalam struktur perulangan, dan apa yang akan terjadi jika variabel ini tidak diubah di setiap iterasi?

Hint:

- Diskusikan risiko *infinite loop* (perulangan tanpa akhir) jika variabel kontrol tidak berubah.
- Bahas pentingnya update variabel (misalnya `i++` dalam `for` atau `while`).
- Tanyakan bagaimana cara mendeteksi dan menghentikan infinite loop.

Pertanyaan Diskusi:

Bagaimana cara menulis perulangan yang efisien dan mudah dibaca pada program C? Apa praktik terbaik dalam memilih jenis loop dan menulis blok kode di dalamnya?

Hint:

- Diskusikan tips penulisan: gunakan `for` jika iterasi terhitung, `while` jika bersyarat logis, `do-while` untuk minimal sekali jalan.
- Bahas pentingnya indentasi, komentar, dan penggunaan blok kurung `{ }` meskipun hanya satu statement.
- Tanyakan pengalaman mahasiswa dalam menulis loop yang nested (bersarang) dan cara menjaga kerapihan kode.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (Perulangan (Looping) di Bahasa Pemrograman C: [Youtube](#)).

Belajar Mandiri



Pelajari secara mandiri terkait Pengenalan AI pada:

- Gonzalez, A. J., & Gonzalez, A. J. (2020). Repetition Structures. *Computer Programming in C for Beginners*, 47-64.

4.3 Break Vs. Continue

Break merupakan instruksi yang menyebabkan proses keluar dari instruksi for, while, do-while, switch.

Continue dapat menyebabkan proses mengabaikan (melewati) instruksi selanjutnya dan melanjutkan proses iterasi berikutnya pada instruksi for, while, do-while.

Berikut contoh program continue pada bahasa C:

```
#include <stdio.h>
int main() {
    int x;
    for(x=1; x<=10; x++) {
        if (x == 5) continue;
        printf("%d ", x);
    }
    return 0;
}
```

Output : 1 2 3 4 6 7 8 9 10

Pada program tersebut terlihat bahwa ketika x bernilai 5 maka proses 5 akan dilewati dan tidak di cetak. Sehingga akan menghasilkan output : 1 2 3 4 6 7 8 9 10

✓ break

- **Definisi:**

break adalah instruksi atau perintah dalam pemrograman yang digunakan untuk **menghentikan eksekusi blok kode tertentu secara paksa** sebelum kondisi normalnya tercapai.

- **Cara kerja:**

Saat break dieksekusi:

- Program **langsung keluar** dari lingkup perulangan (for, while, do-while) atau percabangan switch.
- Eksekusi dilanjutkan ke instruksi setelah blok tersebut.

- **Contoh penggunaan:**

```
for (int i = 0; i < 10; i++) {
    if (i == 5) {
        break; // Keluar dari loop saat i = 5
    }
    printf("%d\n", i);
}
```

👉 Dalam contoh di atas, loop akan berhenti ketika i mencapai 5, meskipun kondisi i < 10 masih benar.

✓ continue

- **Definisi:**

`continue` adalah instruksi yang digunakan untuk **melewati sisa instruksi dalam satu siklus iterasi dan langsung melanjutkan ke iterasi berikutnya** dari perulangan.

- **Cara kerja:**

Saat `continue` dijalankan:

- Instruksi yang berada **setelah `continue` di dalam blok loop akan dilewati.**
- Program melompat ke evaluasi kondisi perulangan untuk melanjutkan iterasi berikutnya.

- **Contoh penggunaan:**

```
for (int i = 0; i < 10; i++) {
    if (i % 2 == 0) {
        continue; // Lewati angka genap
    }
    printf("%d\n", i);
}
```

👉 Dalam contoh ini, angka genap akan dilewati sehingga hanya angka ganjil yang dicetak.

🔑 Perbedaan Inti

Aspek	<code>break</code>	<code>continue</code>
Fungsi	Menghentikan loop atau switch sepenuhnya	Melewati instruksi berikutnya dalam iterasi, tetapi loop tetap berjalan
Lokasi Penggunaan	<code>for</code> , <code>while</code> , <code>do-while</code> , <code>switch</code>	<code>for</code> , <code>while</code> , <code>do-while</code>
Dampak	Langsung keluar dari blok	Langsung lanjut ke iterasi berikutnya

Bahan Diskusi



Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

Pertanyaan Diskusi:

Jelaskan skenario praktis kapan `break` lebih tepat digunakan dibandingkan `continue` dalam perulangan, dan sebaliknya. Berikan contoh situasi nyata yang relevan.

Hint:

- Pikirkan kasus `break` → Misalnya, ketika ingin menghentikan pencarian begitu data ditemukan agar tidak boros waktu.
- Pikirkan kasus `continue` → Ketika hanya ingin melewati data tertentu yang tidak perlu diproses tanpa menghentikan loop seluruhnya.
- Coba hubungkan dengan penggunaan `break` di `switch` dan `continue` yang tidak ada di `switch`.

Pertanyaan Diskusi:

Apa yang akan terjadi pada alur program jika programmer keliru menggunakan `break` padahal seharusnya `continue` (atau sebaliknya)? Jelaskan dengan contoh kasus yang mungkin terjadi pada loop perhitungan atau validasi data.

Hint:

- Pikirkan jika `break` digunakan di program yang menghitung total penjumlahan — apa yang terjadi kalau loop berhenti lebih cepat dari seharusnya?
- Pikirkan jika `continue` digunakan padahal harusnya `break` — program bisa jalan terus padahal kondisi berhenti sudah terpenuhi.
- Coba diskusikan dampaknya pada performa dan hasil output.

Pertanyaan Diskusi:

Buatlah logika loop yang menggunakan `break` dan `continue` secara bersamaan. Apa tujuan penggabungan ini, dan bagaimana alurnya agar tidak membingungkan?

Hint:

- Coba bayangkan program yang melewati nilai tertentu (dengan `continue`), tapi juga akan berhenti total kalau menemukan nilai tertentu (dengan `break`).
- Misalnya: memproses angka 1–20, lewati kelipatan 3, tapi berhenti total jika menemukan angka 17.
- Diskusikan urutan evaluasi kondisi: mana yang perlu dicek lebih dulu?

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (break and continue in C: [Youtube](#)).

4.4

Go to dan Label

Bahasa C termasuk salah satu bahasa pemrograman tingkat menengah yang mendukung berbagai struktur kontrol alur program, seperti percabangan, perulangan, dan pernyataan lompatan. Salah satu fitur lompatan eksplisit yang disediakan adalah `goto`, yang memungkinkan program melompat langsung ke bagian lain dalam blok kode dengan menggunakan label sebagai penanda tujuan. Konsep `goto` ini mirip dengan cara kerja label dan instruksi lompat pada bahasa rakitan atau Assembly, sehingga memberikan fleksibilitas bagi programmer untuk memindahkan eksekusi ke titik tertentu secara langsung.

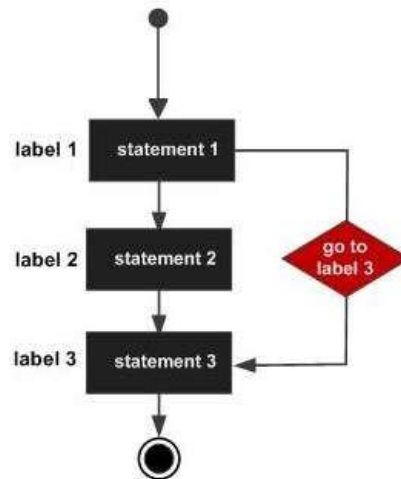
Meskipun demikian, penggunaan `goto` dalam praktik pemrograman modern sangat jarang direkomendasikan. Alasannya adalah karena `goto` dapat memutus alur kontrol program yang terstruktur, sehingga menyebabkan kode sulit dibaca dan dipelihara. Ketika `goto` digunakan berulang-ulang dalam program yang besar, alur eksekusi menjadi tidak linear dan berpotensi menimbulkan apa yang disebut dengan *spaghetti code* — yaitu kode yang saling terkait tidak beraturan seperti mie yang kusut.

Selain itu, kesalahan penempatan atau pemahaman label tujuan pada `goto` bisa menimbulkan bug yang sulit dilacak. Misalnya, program dapat melompati inisialisasi variabel, melewati bagian penting dari logika perulangan, atau bahkan menciptakan jalur eksekusi tak terduga yang bisa menyebabkan program berhenti tiba-tiba atau berperilaku tidak semestinya. Oleh karena itu, debugging pada program yang bergantung pada `goto` cenderung memakan waktu lebih lama dibandingkan struktur kontrol alur yang lebih terstruktur seperti `if-else`, `switch`, dan `loop`.

Meski sering dianggap "berbahaya", `goto` tidak sepenuhnya tidak berguna. Dalam beberapa kasus yang sangat spesifik, seperti keluar dari beberapa tingkat nested loop secara langsung atau menangani pembersihan memori dalam blok `error handling`, `goto` dapat menjadi solusi yang lebih efisien daripada harus membuat flag atau kondisi tambahan. Namun, penggunaannya pun tetap harus dilakukan dengan penuh kehati-hatian dan biasanya dilengkapi dengan dokumentasi yang jelas untuk meminimalkan potensi kekeliruan.

Sebagai kesimpulan, meskipun Bahasa C menyediakan instruksi `goto` sebagai salah satu kontrol alur, para programmer modern dianjurkan untuk menghindarinya sebisa mungkin. Alih-alih menggunakan `goto`, praktik terbaik adalah memanfaatkan struktur perulangan (`for`, `while`, `do-while`) dan percabangan (`if-else`, `switch`) yang lebih terstruktur dan mudah diikuti. Dengan cara ini, program akan memiliki alur logika yang jelas, lebih mudah dibaca, diuji, serta dirawat oleh programmer lain di masa mendatang.

Berikut bentuk flow chart dari `goto` label pada bahasa C :



Berikut contoh penggunaan goto label pada bahasa C:

```

#include <stdio.h>

int main ()
{
    /* local variable definition */
    int a = 10;

    /* do loop execution */
    LOOP:do
    {
        if( a == 15)
        {
            /* skip the iteration */
            a = a + 1;
            goto LOOP;
        }
        printf("value of a: %d\n", a);
        a++;
    }while( a < 20 );

    getchar();
    return 0;
}
  
```

```

value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 16
value of a: 17
value of a: 18
value of a: 19
  
```

Bahan Diskusi



Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

Pertanyaan Diskusi:

Dalam situasi seperti apa penggunaan `goto` masih dianggap wajar atau lebih efisien dibandingkan menggunakan struktur kontrol alur yang lain?

Hint:

- Diskusikan contoh spesifik seperti: keluar cepat dari nested loop yang dalam, error handling yang kompleks, atau pembersihan resource di blok `cleanup`.
- Bandingkan dengan alternatif tanpa `goto` — misalnya dengan flag, break multi level, atau fungsi.
- Tanyakan pengalaman mahasiswa: apakah pernah melihat atau membuat program yang memakai `goto`?

Pertanyaan Diskusi:

Apa risiko utama jika kita terlalu sering menggunakan `goto` dalam program? Mengapa `goto` sering dihubungkan dengan munculnya *spaghetti code*?

Hint:

- Bahas bagaimana `goto` bisa membuat alur program melompat-lompat dan sulit ditelusuri.
- Jelaskan bahwa label yang tidak terstruktur dapat melompati deklarasi variabel, inisialisasi, atau logika penting.
- Ajak peserta diskusi untuk membayangkan debugging kode panjang yang banyak `goto` dibanding kode terstruktur.

Pertanyaan Diskusi:

Jika disarankan untuk menghindari `goto`, struktur kontrol apa yang sebaiknya digunakan untuk mengganti peran `goto` dalam pengembangan program?

Hint:

- Bahas bagaimana loop (`for`, `while`, `do-while`) bisa menggantikan perulangan dengan jalur kontrol yang rapi.
- Diskusikan bagaimana `break` dan `continue` bisa digunakan untuk mengatur alur dalam loop.
- Tekankan penggunaan fungsi untuk modularisasi logika, sehingga tidak perlu `goto` untuk mengatur jalur keluar program.

4.5

Error Type

Terdapat beberapa jenis error yang sering terjadi pada umumnya, antara lain:

- a. **Lupa untuk memberikan tanda kurung batas blok if.** Perlu diingat bahwa Anda boleh menghilangkan penulisan “{ “ }” sebagai penanda pada if statement jika hanya terdapat satu statement pada blok tersebut. Lihat contoh berikut:

```
if(radius >=0 )
    area = radius * radius * PI;
    System.out.println("The area is " + area);
```

Pada potongan program diatas, bila syarat radius ≥ 0 terpenuhi maka program akan menjalankan statement if yaitu `area = radius * radius * PI`. Kemudian program akan melanjutkan untuk mengeksekusi statement berikutnya (dalam konteks melanjutkan pengekseskusion program yang berada diluar blok if. Bila syarat radius ≥ 0 tidak terpenuhi, maka program tidak akan menjalankan perhitungan area, tetapi akan langsung mengeksekusi statement `System.out.println`, karena statement tersebut tidak termasuk statement didalam blok if.

- b. **Memberikan tanda “;” pada akhir persyaratan if.** Hal ini akan menyebabkan error. Ingat bahwa tanda titik koma “;” digunakan untuk menandakan akhir dari sebuah statement.

```
if(radius >=0 ;) —————> Logic error
{
    area = radius * radius * PI;
    System.out.println("The area is " + area);
}
```

- c. **Perulangan dalam pengecekan nilai.** Pada contoh dibawah ini terlihat bahwa statement if akan dijalankan bila syarat/kondisi bernilai benar/true. Maka secara tersirat if akan melakukan pengecekan apakah nilai dari variable yang dicek bernilai benar.

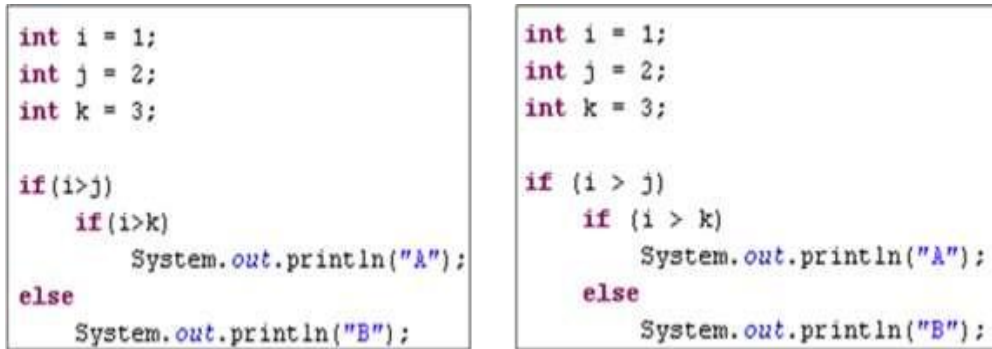
```
if (even == true)
    System.out.println("It is even");
```

Equivalent

```
if (even)
    System.out.println("It is even");
```

This is better

- d. Pernyataan blok else yang ambigu dalam nested if. Contoh:



```

int i = 1;
int j = 2;
int k = 3;

if(i>j)
    if(i>k)
        System.out.println("A");
else
    System.out.println("B");
  
```

```

int i = 1;
int j = 2;
int k = 3;

if (i > j)
    if (i > k)
        System.out.println("A");
else
    System.out.println("B");
  
```

Anda dapat melihat blok else pada kolom sebelah kiri cukup ambigu, apakah blok else tersebut merupakan blok else milik if(i>j) atau if(i>k). Seharusnya blok else tersebut ditulis seperti potongan program disebelah kanan. Bila Anda ingin membuat else untuk milik if(i>j), maka Anda perlu memberikan tanda “{ }” untuk membatasi statement blok if(i>j) tersebut.

- e. **Compile time-error** : disebabkan oleh kesalahan sintaks.
- f. **Link-time error** : berhasil di compile, namun menyebabkan link-error, dikarenakan kesalahan pada saat kompilasi. Biasanya disebabkan karena file .exe tidak berhasil dihubungkan sehingga menyebabkan format file .exe tidak dapat dibentuk.
- g. **Run-time error** : program berhasil di susun namun terdapat kesalahan pada saat run-time. Biasanya hal ini disebabkan karena operasi numerik seperti pembagian dengan nol, dan lain sebagainya.
- h. **Logical error** : berhasil di eksekusi namun, hasil output tidak sesuai dengan yang diharapkan. Hal ini terjadi dikarenakan kesalahan pada algoritma yang dibentuk

4.6 Sesi Pemecahan Masalah dengan AI: Debugging Kontrol Program

HAICOLA Section: Mahasiswa dapat menggunakan AI Code Assistant (seperti **Gemini** atau **ChatGPT**) untuk melakukan pengecekan sintaks, memahami error, dan memperbaiki kode C secara efisien.

Salah satu tantangan terbesar dalam pemrograman C adalah mengatasi kesalahan logika dalam struktur kontrol, seperti perulangan (*loops*) dan pernyataan bersyarat (*conditional statements*). Sesi ini menjelaskan bagaimana Anda dapat menggunakan **AI Code Assistant** (misalnya, Gemini) sebagai alat bantu untuk menemukan dan memahami kesalahan umum, yang secara langsung mendukung LO 2: Analyzing a program to solve problems using the C language.

Fokus: Kesalahan Logika dalam Loops dan Conditionals

Kesalahan logika sering terjadi ketika kode berjalan tanpa *crash*, tetapi menghasilkan output yang salah. Dua area masalah utama dalam mata kuliah ini adalah:

1. **Struktur Perulangan (*Loops*: *for*, *while*):**

- **Infinite Loops:** Terjadi ketika kondisi perulangan tidak pernah menjadi *false* (misalnya, variabel kontrol tidak pernah diperbarui, atau kondisi awalnya salah).
- **Off-by-One Errors (OBOE):** Terjadi ketika perulangan berjalan satu iterasi terlalu banyak atau terlalu sedikit, biasanya karena kondisi batas (< vs. <=) salah.
- **Kesalahan Array:** Perulangan yang mencoba mengakses elemen di luar batas yang ditentukan (misalnya, mencoba mengakses `array[N]` pada *array* berukuran *N*).

2. **Pernyataan Bersyarat (*Conditionals*: *if-else*, *switch*):**

- **Kesalahan Prioritas:** Menggunakan operator relasional atau logika yang salah (&&, ||) atau salah memahami urutan eksekusi.
- **Assignment vs. Comparison:** Kesalahan klasik di C adalah menggunakan operator *assignment* (=) di dalam pernyataan *if* ketika yang dimaksud adalah operator perbandingan (==).

Contoh Praktis: Menggunakan AI untuk *Debugging*

Anda dapat menggunakan AI Code Assistant (ChatGPT/Gemini) untuk membantu Anda atau teman Anda mengidentifikasi dan menjelaskan kesalahan ini.

Skenario 1: *Infinite Loop*

Kode yang Bermasalah:

```
#include <stdio.h>

int main() {
    int i = 1;
    while (i <= 5) {
        printf("Angka: %d\n", i);
        // Error: Variabel 'i' tidak pernah di-increment
    }
    return 0;
}
```

Permintaan (Prompt) ke AI:

"Kode C saya di bawah ini menghasilkan infinite loop. Jelaskan mengapa dan berikan kodenya yang sudah diperbaiki."

Respon AI yang Diharapkan:

AI akan menjelaskan bahwa variabel i di dalam loop `while` tidak pernah bertambah, sehingga kondisi $i \leq 5$ akan selalu benar. AI kemudian memberikan kode yang diperbaiki dengan menambahkan $i++$.

Skenario 2: Off-by-One Error

Kode yang Bermasalah (C):

```
#include <stdio.h>
#define N 5

int main() {
    int data[N] = {10, 20, 30, 40, 50};
    // Perulangan ini berjalan dari i=0 sampai i=5, yang melebihi batas array
    for (int i = 0; i <= N; i++) {
        printf("Element %d: %d\n", i, data[i]);
    }
    return 0;
}
```

Permintaan (Prompt) ke AI:

"Saya mencoba mencetak 5 elemen array, tetapi program saya mengalami segmentation fault atau mencetak nilai yang salah. Array saya berukuran 5 (indeks 0 hingga 4). Ini kodenya: [masukkan kode di atas]. Apa yang salah dengan kondisi loop saya?"

Respon AI yang Diharapkan:

AI akan menunjukkan bahwa data berukuran 5, yang berarti indeks yang valid adalah 0 hingga 4. Kondisi $i \leq N$ memungkinkan $i=5$, yang merupakan akses di luar batas array (Off-by-One Error). Solusi perbaikannya adalah mengubah kondisi menjadi $i < N$.

Praktik Terbaik untuk Mahasiswa

- **Jangan Copy-Paste:** Gunakan AI untuk **memahami** kesalahan, bukan sekadar mendapatkan solusi. Peran Anda adalah *menganalisis* (LO 2).
- **Berikan Konteks:** Selalu sertakan deskripsi masalah, *error message*, dan **kode yang lengkap** dalam *prompt* Anda untuk mendapatkan hasil terbaik.
- **Periksa Employability Skill:** Latihan *debugging* dengan AI membantu mengasah kemampuan **Problem-Solving and Decision-Making**, yang merupakan kompetensi lulusan.

Bahan Diskusi



Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

Pertanyaan Diskusi:

Mengapa penting untuk selalu menggunakan tanda kurung kurawal `{ }` pada blok `if` meskipun hanya ada satu statement? Diskusikan konsekuensi logika jika tanda kurung dihilangkan pada program bercabang.

Hint:

- Pikirkan bagaimana compiler C membaca perintah baris per baris.
- Diskusikan kemungkinan perintah di luar `if` dieksekusi tanpa syarat.
- Bagikan pengalaman jika pernah mengalami bug karena hal ini.

Pertanyaan Diskusi:

Jelaskan perbedaan **compile-time error**, **link-time error**, **run-time error**, dan **logical error**. Berikan contoh nyata pada coding C sederhana, bagaimana error tersebut muncul, dan bagaimana cara mengatasinya.

Hint:

- Bedakan error yang terjadi saat mengetik kode (syntax error) vs error saat program dijalankan.
- Coba sebut contoh: file hilang saat linking, pembagian dengan nol saat run-time, dan logika salah yang hasilnya tak sesuai.

Pertanyaan Diskusi:

Mengapa `else` dalam *nested if* bisa menimbulkan kebingungan? Diskusikan pentingnya penggunaan kurung `{ }` pada nested `if-else` agar flow program jelas dan mudah dibaca.

Hint:

- Jelaskan bagaimana *indentasi* tidak berpengaruh pada compiler C.
- Buat mini contoh: `if` di dalam `if`, lalu `else` tanpa `{ }` — tanyakan ke teman, `else` ini milik `if` yang mana?
- Diskusikan bagaimana penulisan rapi dan blok yang jelas membantu debugging.

Post-Test

Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

Soal 1: (Topik: Pemilihan - Selection)

Perhatikan potongan kode berikut ini:

```
int nilai = 75;
if (nilai >= 80) {
    printf("A");
} else if (nilai >= 60) {
    printf("B");
} else {
    printf("C");
}
```

Apa output dari program tersebut?

- A. A
- B. B
- C. C
- D. Tidak ada output

Soal 2: (Topik: Perulangan - Repetition)

Manakah pernyataan yang BENAR tentang perulangan do...while?

- A. Kondisi diperiksa sebelum blok statement dijalankan.
- B. Blok statement dijalankan minimal satu kali, lalu kondisi diperiksa.
- C. Tidak dapat digunakan untuk perulangan tanpa batas.
- D. Harus selalu digunakan dengan switch-case.

Soal 3: (Topik: Break vs. Continue)

Perhatikan potongan kode berikut ini:

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }
    printf("%d ", i);
}
```

Apa output dari program tersebut?

- A. 1 2 3 4 5
- B. 1 2 4 5
- C. 1 2 3 5
- D. 1 2 3 4

Post-Test



Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

Soal 4: (Topik: Goto dan Label)

Mengapa penggunaan `goto` sebaiknya dihindari dalam praktik pemrograman modern?

- A. Karena membuat program lebih cepat.
- B. Karena dapat menghasilkan *spaghetti code* yang sulit dipelihara.
- C. Karena menggantikan fungsi `break` sepenuhnya.
- D. Karena hanya dapat digunakan di luar loop.

Soal 5: (Topik: Error Type)

Manakah contoh **logical error** pada program C berikut ini?

- A. Syntax error pada tanda kurung.
- B. Penggunaan variable tanpa deklarasi.
- C. Loop yang berjalan tanpa henti karena kondisi salah.
- D. Kompiler gagal men-generate file output.

Kesimpulan

- 1 Fungsi seleksi digunakan untuk mengaplikasikan alternatif pilihan. Ada beberapa jenis seleksi: if statement, if-else statement, nested if statement, switch statement, dan conditional expression.
- 2 Semua macam dari if statement merupakan struktur control yang berdasarkan pada expression. Bergantung pada nilai true atau false yang dihasilkan dari expression tersebut (syarat seleksi), seleksi if ini akan memilih satu dari dua pilihan.
- 3 Switch statement adalah struktur control yang bergantung pada tipe dari switch- expression yang hanya dapat mengaplikasikan tipe char, byte, short, atau int.
- 4 Keyword break bersifat pilihan (optional), tetapi biasanya digunakan pada setiap akhir case statement dengan tujuan untuk memberhentikan pengeksekusi program yang tersisa dalam switch statement. Jika break statement tidak digunakan, maka case statement yang selanjutnya tetap akan dieksekusi, dimana switch statement melakukan pengeksekusi secara sequential order.
- 5 Ada 3 jenis sintaks perulangan yang dapat digunakan, yaitu while, do-while dan for.
- 6 Break merupakan instruksi yang menyebabkan proses keluar dari instruksi for, while, do-while, switch.
- 7 Continue dapat menyebabkan proses mengabaikan (melewati) instruksi selanjutnya dan melanjutkan proses iterasi berikutnya pada instruksi for, while, do-while.
- 8 Bahasa C menyediakan cara untuk menyediakan lompatan goto melalui label dengan fungsi yang sama.
- 9 Penggunaan goto ini sangat tidak dianjurkan dalam bahasa pemrograman dikarenakan akan membuat struktur code menjadi tidak teratur, dan membuat sulit untuk melacak aliran kontrol dari sebuah program.
- 10 Beberapa macam tipe error pada bahasa C :
 - Compile time-error
 - Link-time error
 - Run-time error
 - Logical error

Daftar Pustaka

- Paul J. Deitel & Harvey. Deitel. (2022). C how to program. 09. Pearson Education. Hoboken. ISBN:978-0-13-739839-3. Chapter 3 & 4
- Choosing between Alternatives:
<http://docs.roxen.com/pike/7.0/tutorial/statements/conditions.xml>
- Getting Controls: <http://aelinik.free.fr/c/ch10.htm>
- Doing the Same Thing Over and Over: <http://aelinik.free.fr/c/ch07.htm>
- https://www.tutorialspoint.com/cprogramming/c_goto_statement.htm
- Gonzalez, A. J., & Gonzalez, A. J. (2020). Repetition Structures. Computer Programming in C for Beginners, 47-64.

Berikut jawaban untuk pertanyaan Post-Test diatas:

1. **Jawaban:** B

Alasan: Nilai 75 lebih besar atau sama dengan 60 tetapi kurang dari 80, maka blok `else if` akan dijalankan, mencetak "B".

2. **Jawaban:** B

Alasan: Pada `do...while`, blok statement selalu dijalankan minimal sekali sebelum kondisi dicek.

3. **Jawaban:** B

Alasan: Saat `i == 3`, `continue` dilewati, jadi 3 tidak dicetak, hanya 1 2 4 5.

4. **Jawaban:** B

Alasan: `goto` memecah alur terstruktur dan membuat logika program sulit dibaca, menyebabkan potensi *spaghetti code*.

5. **Jawaban:** C

Alasan: Loop yang tidak berhenti terjadi karena kesalahan logika (logical error), bukan syntax atau runtime error.